团队技术博客 · 需求定稿 v1.0

> 占位名：TeamBlog（上线前团队投票定名，只影响首页标题、Logo、域名三处）
>
> 本文件是 v1.0 原文归档，**不做修订**。所有补全与修正记录在 02-实施方案-v1.1.md。

1. 角色与权限

角色	数量	登录	权限
管理员	4 名，权限完全对等	JWT	全部功能：成果上传、编辑任意成员个人页、成员管理（邀请/移除/角色变更）、打卡、学习内容上传、查看留言反馈、删除任意成果
成员	邀请制，不限	JWT	成果上传（直接展示，无审核）、编辑自己个人页、学习内容上传、打卡、评论
访客	不限	无需登录	浏览团队主页/个人页/成果、对任意项目发起 AI 对话；评论需登录，访客不可评论

JWT 要点：

- Access Token 30 分钟，Refresh Token 7 天
- payload 携带 `role`
- 每个写接口后端独立校验角色，不依赖前端隐藏按钮

2. 功能清单

2.1 团队主页（访客可见）

- 团队名称 + 简介 + 团队公共邮箱（固定展示）
- 学习打卡统计：GitHub 风格热力图，展示团队整体打卡密度，支持按成员筛选
- 积分等级展示：成员卡片显示等级徽章，供访客感知团队活跃度
- 成果流：项目 / 证书 / 奖项混合列表，最新在前
- 成员卡片列表：头像、等级、最近学习标题

2.2 个人页 `/u/{username}`

- 头像、简介、GitHub 链接
- 团队公共邮箱（固定）+ 个人邮箱（由成员开关 `email_public` 控制，默认关闭；开启时页面显示一行"公开后访客可见，可能被收集"）
- 最近学习内容：最近 5 条，支持 Markdown 上传、代码文件上传（代码转 Markdown 代码块展示）
- 最近登录时间：登录时自动更新
- 个人打卡热力图

- 个人成果列表
- 编辑权限：成员编辑自己，管理员编辑所有人

2.3 成果模块（项目 / 证书 / 奖项统一抽象）

类型	必填字段
项目	标题、描述、GitHub 链接、项目内容（手动粘贴 README 关键内容）
证书	标题、描述、图片/链接
奖项	标题、描述、图片/链接

- 成员上传后直接展示，无审核状态
- 管理员保留删除/下架任意成果的权限
- 项目链接失效由上传者负责，管理员可下架

2.4 打卡与积分

积分规则（写死在后端，不做管理员配置）：

行为	积分
每日打卡	+1
上传学习内容（Markdown/代码）	+10
上传成果（项目/证书/奖项）	+15
连续打卡 7 天	+5（只加一次）
连续打卡 14 天	+10（只加一次）
连续打卡 30 天	+20（只加一次）

- **断签规则**：断一天，连续天数归零，已获得积分不扣。
- **验证**：30 天只打卡 = 30 分；每周认真写 2 篇 × 4 周 = 80 分。内容型成员积分是打卡型的 2.6 倍，符合"认真写内容者高于只打卡者"的要求。

等级（只展示，不挂钩权限）：

等级	累计积分
Lv1	0–49
Lv2	50–149
Lv3	150–299
Lv4	300–599
Lv5	600+

勋章：一期不做，二期加"勋章定义 + 授予记录"两张表。

2.5 AI 功能

场景	触发	输入	输出
访客问项目	访客点任意项目 → 对话	仅当前项目的标题 + 描述 + 项目内容（手动粘贴的 README 文本）+ 访客问题	流式回答
成员学习总结	成员上传学习内容后点"生成总结"	该篇 Markdown/代码内容	100–200 字摘要，成员可编辑后保存

- 访客问答不检索全站，不需要向量库 / RAG
- 项目内容来源：先 A 后 B——一期手动粘贴 README 关键内容；二期加 GitHub API 自动抓取 + "刷新 README"按钮，手动内容保留为兜底
- 访客 AI 对话接口需 IP 限流（建议每分钟 10 次），防刷 API 额度

2.6 评论与留言

- **评论**：仅登录用户可发表，挂在成果或学习内容下，一级评论 + 回复，不做无限嵌套
- **留言反馈**：访客和成员均可提交（访客填邮箱，不强制登录），管理员后台查看，与内容评论分开存储

---

3. 数据模型（核心表）

```
users
  id, username, email, password_hash, role(admin/member),
  avatar_url, bio, github_url, email_public(bool, default false),
  total_points, last_login_at, created_at

projects
  id, owner_id, type(project/certificate/award),
  title, description, github_url, content_md, image_url, created_at

learning_logs
  id, user_id, title, content_md, code_url,
  type(note/code), log_date, created_at

checkins
  id, user_id, checkin_date, note
  -- (user_id, checkin_date) 唯一索引

comments
  id, user_id, target_type(project/learning_log), target_id,
  parent_id(可空), content, created_at

feedbacks
  id, name, email(可空), content, created_at
  -- 留言反馈，管理员后台查看

ai_conversations
  id, visitor_ip, project_id(可空), messages_json, created_at
```

设计决策：

- 项目、证书、奖项统一在 `projects` 表用 `type` 区分，前端一个列表接口，后端一套 CRUD
- `learning_logs` 与 `checkins` 解耦——打卡只记"今天学了"，学习日志记具体内容，积分和热力图都从 `checkins` 算

---

4. 技术栈

层	技术
后端 + 前端	Next.js（App Router）+ TypeScript，全栈同仓库
ORM	Prisma
数据库	SQLite（开发）→ PostgreSQL（生产）
认证	NextAuth（Credentials + JWT）
AI	OpenAI 兼容接口（阿里云百炼 qwen-plus / DeepSeek），环境变量配置
部署	Docker Compose + Nginx + 1Panel，2 核 4G 云服务器

---

5. 前端模板选型

约束：从 GitHub 找现成开源模板改；技术栈冲突时为了后端放弃模板（即模板必须在 Next.js 系内选）。

候选：

仓库	特点
Taxonomy（shadcn 官方示例）	Next.js App Router + NextAuth + Prisma + Tailwind，结构与需求几乎重合，首选
next-ai-blog-cms	Next.js 全栈博客 CMS，含 Markdown、认证
blog-fullstack-studio	前后端一体博客脚手架，需确认维护活跃度

选型标准：

1. Next.js App Router + TypeScript
2. 自带 Auth（NextAuth 或等价）+ Prisma/数据库示例
3. 许可证 MIT 或 Apache 2.0
4. 近 3 个月有提交
5. Star ≥ 500（参考，不硬性）

- **责任人**：前端负责人 + 后端负责人，2 天内定 1 个，产出"保留哪些页面、删哪些、加哪些"的一页清单。
- **风险提示**：模板改出的部分，二期重写时会有成本。现在就在模板代码里标注"临时借用，二期重写"，避免越改越深。

---

6. 部署

- 云服务器：2 核 4G，Ubuntu 22.04，40G 系统盘
- 安装 1Panel，Docker Compose 编排 web（Nginx）/ api（Next.js）/ db（PostgreSQL）
- Nginx 配 HTTPS（1Panel 一键 Let's Encrypt）
- 每天凌晨 `pg_dump` 备份到另一目录或对象存储（打卡和积分数据丢失不可恢复）

关键环境变量：

```
DATABASE_URL=postgresql://user:pass@db:5432/blog
JWT_SECRET=（长随机字符串）
OPENAI_BASE_URL=https://dashscope.aliyuncs.com/compatible-mode/v1
OPENAI_API_KEY=sk-...
OPENAI_MODEL=qwen-plus
```

上线检查清单：

1. `/api/health` 返回 200
2. 访客能打开团队主页，热力图有数据
3. 成员登录后能打卡、上传 Markdown、个人页更新
4. 访客点项目能触发 AI 对话并流式返回
5. 管理员后台能删除成果、管理成员、查看留言

---

7. 一期 / 二期范围

一期（MVP）：

- JWT 登录、三种角色权限
- 团队主页（热力图、等级展示、成果流、公共邮箱）
- 个人页（简介、最近学习、最近登录、个人邮箱开关、个人热力图）
- 成果上传（直接展示）+ 管理员删除
- 打卡 + 积分 + 等级
- 学习内容 Markdown/代码上传
- 评论（登录后）+ 留言反馈（管理员查看）
- AI 访客问答（手动粘贴项目内容）
- AI 成员学习总结
- 部署上线

二期：

- AI 项目内容 GitHub API 自动抓取（B 方案，手动内容保留为兜底）
- 勋章系统
- 成果链接失效举报入口
- 前端模板临时部分重写
- pgvector RAG（若项目数量增长到需要全站检索时再评估）

---

8. 分工建议（4 名管理员）

角色	主责
后端 + 数据库 + 部署	API、Prisma 模型、JWT 中间件、Docker Compose、备份
前端	模板改造、页面与交互、热力图组件、Markdown 渲染
AI	两个 AI 场景、限流、prompt 设计、对话记录
测试 + 内容运营	测试用例、上线检查、项目/证书录入、成员支持、文档

四人交叉 review，主责清晰。

---

9. 两个必须守住的原则

1. **AI 不是第一个迭代的事。** 登录、内容、打卡、部署跑通后再接 AI。
2. **权限校验必须在后端。** 前端隐藏按钮只是体验优化，每个写接口独立校验 `role`，否则成员抓包就能调管理员接口。